

Estructuras de Datos — Quiz 03

Dynamic Arrays and Amortized Analysis · analysis.pdf · Semestre 2026-2

Nombres: Kevin Javier Gonzalez, Iván Felipe Maluche Suarez, Angelica Pascagaza Vega **Documento:** 1093432420 **Grupo:** 3

Notación y modelo de costo

P1 — Arreglo dinámico desde cero

a) **Implementación.** En el archivo de `DynamicArray.java` se encuentra la implementacion del arreglo dinamico.

b) **Invariantes.**

I1.

$$0 \leq size \leq capacity$$

I2.

$$capacity \geq 1$$

I3.

$$index \in [0, size), data[size] == null$$

c) **Complejidad worst-case.**

Operación	Worst-case	Justificación
get	$\Theta(1)$	El metodo solamente comprueba si el index es valido y devuelve el dato, debido a que el array por defecto ya lo hace sin necesidad de recorrer nada cuesta $O(1)$
set	$\Theta(1)$	Similar a get, es una busqueda de costo 1 mas en este caso se asigna al valor un dato manteniendo constante la cota
append	$O(n)$	Debido al resize en el peor caso es necesario construir un nuevo array y dato por dato del anterior asignarlo en el mismo.
removeLast	$\Theta(1)$	Tal como el caso de get y set consiste en guardar el ultimo valor, asignarle un nulo y disminuir size, siendo operaciones que me dan una cota constante.

d) **Worst-case vs. amortizado.**

Al realizar el append siempre que $size < a\ capacity$ el coste de operacion sera "barata", lo cual permite que cuando pase el caso en que $(size = capacity)$ el costo de resize se amortizara en el total.

P2 — Snake sobre DynamicArray con desplazamiento

a) **Costo de removeTail.**

Al llamar removeTail en esa implementacion es necesario eliminar la cola y luego reasignar los valores que quedan una posicion hacia atras, esto implica un costo $O(n)$ (con n siendo en este caso el size) al tener que recorrer el array .

b) **Costo total de m movimientos.**

Al ser de longitud k habria un costo de $O(k)$ lo cual al realizarse m veces nos daria un costo total de $O(km)$.

c) **Validez de la afirmación.** Esto es falso, `move()` no es amortizado porque en cada operacion de `move()` tengo que hacer n (size) actualizaciones de posicion de la serpiente lo que hace que en promedio

sea $\frac{O(n \cdot n)}{n} \rightarrow O(n)$ veces el costo amortizado, a diferencia del caso de append donde gracias a la distribución entre operaciones caras y baratas el coste se amortigua.

d) Bottleneck.

El bottleneck en esta implementación consistiría en que debido a que en cada movimiento que la serpiente no consuma una manzana (lo cual pasara frecuentemente) será necesario ejecutar un RemoveTail, el cual como se explico en el punto anterior es $O(n)$. Esto culminara en que esta operación básica y siempre necesaria para todo el loop jugable tenga un costo considerable y pueda generar problemas de estabilidad al ejecutarse en casi todo momento.

P3 — Rediseño: representación interna

a) Variables de estado.

- data:
- start:
- size:
- capacity:

b) Mapeo lógico $\rightarrow$ físico.

$$\text{phys}(i) =$$

c) Operaciones.

- addHead:
- removeTail:
- get:

d) resize.

e) Invariantes y corrección.

S1.

S2.

S3.

S4.

f) Traza.

Mov.	Arreglo físico	start	size	capacity	¿resize?	Orden lógico
inicial						
N1 (no come)						
N2 (come)						
N3 (no come)						
N4 (no come)						
N5 (come)						
N6 (no come)						

P4 — Análisis amortizado

a) Aggregate Method

La longitud inicial es 2 y cada manzana suma 1, pero como addHead va antes de removeTail, el tamaño pasa transitoriamente de k a $k + 1$. Por eso la longitud máxima transitoria es $n_{\text{máx}} = g + 3 = O(g)$. Con

$C_0 = 2$ al ir duplicando la **capacity**, las capacidades son $2, 4, 8, \dots, 2^{r+1}$. Un **resize** se ejecuta cuando **size** = **capacity**, y en ese momento se copian exactamente C_j elementos. Como la estructura nunca supera $n_{\text{máx}}$, hay $r = O(\log g)$ **resizes** y el total de copias es:

$$\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2 n_{\text{máx}} = O(g).$$

Resultado: el total de copias es lineal en g , es decir $O(g)$.

b) Suma geométrica

La capacidad solo cambia en un **resize**, y entre el **resize** j y el $j + 1$ vale C_j . El **resize** $j \rightarrow j + 1$ se ejecuta cuando **size** = C_j , así que en ese instante el arreglo está lleno y se copian exactamente C_j elementos. Por el proceso de duplicación, esos valores son $2, 4, 8, \dots, 2^r$. La suma es geométrica de razón 2:

$$\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2 \cdot 2^r = 2 C_{r-1}.$$

El último término domina a todos los anteriores, así que la suma es proporcional al último término y no a la cantidad de términos. Como $C_{r-1} = O(g)$, el total de copias es $O(g)$.

c) Costo total y amortizado de addHead

El costo total real de m movimientos se descompone en $O(m)$ por los **addHead** (escritura + índices), $O(m)$ por los **removeTail** (leer, poner **null**, avanzar índice) y $O(g)$ por las copias de los **resizes**. Entonces $T(m) = O(m) + O(g) = O(m + g) = O(m)$, usando $g \leq m$. El costo amortizado de **addHead** es $T(m)/m = O(m + g)/m = O(1)$. Aunque un **addHead** individual cueste $O(n)$ en el peor caso por un **resize**, su costo amortizado es constante.

d) Papel de m y g

m y g miden cosas distintas. m controla el trabajo por operación: cada **MOVE** hace un **addHead** y, si no come, un **removeTail**, ambos $O(1)$; esta es la parte del costo que depende de cuántas operaciones se hacen. g controla el crecimiento: la longitud máxima transitoria es $g + 3$, la capacidad final es $O(g)$, hay $O(\log g)$ **resizes** y $O(g)$ copias; esta es la parte del costo que depende del tamaño alcanzado. La descomposición es $T(m) = O(m) + O(g)$. Como $g \leq m$, el total es $O(m)$, pero las dos fuentes son conceptualmente independientes: con $g = 0$ no hay **resizes**; con $g \approx m$ hay $O(\log m)$ **resizes** pero el costo sigue siendo lineal en m .

e) Método del potencial

Se elige $\Phi(\text{size}, C) = 2 \cdot \text{size} - C$. Justo después de un **resize** (con **size** = C_{old} , $C_{\text{new}} = 2C_{\text{old}}$) el potencial vale $\Phi = 0$; justo antes, con **size** = C , vale $\Phi = C$, suficiente para pagar las C copias. El costo amortizado es $\hat{c} = c + \Delta\Phi$.

Operación	$\Delta\Phi$	$\hat{c}$
addHead sin resize	2	$1 + 2 = 3$
addHead con resize	$2 - C$	$(C + 1) + (2 - C) = 3$
removeTail	-2	$1 - 2 = -1$

La suficiencia del potencial frente a **removeTail** se justifica porque esta operación **no libera capacidad**, así que no ejecuta **resizes** ni copias; solo reduce el potencial en 2, es decir, devuelve crédito sin consumirlo. Como el potencial solo se gasta en los **resizes** (donde $\Delta\Phi \approx -C$) y cada **resize** ocurre cuando $\Phi = C$, el crédito acumulado siempre alcanza. En general, $\sum_{i=1}^t \hat{c}_i = \sum_{i=1}^t c_i + \Phi_t - \Phi_0$, con $\Phi_0 = 2$ y $\Phi_t \leq 2 \cdot \text{size}_t = O(g)$, de donde $\sum c_i = O(m)$, coherente con el método agregado. El costo amortizado por operación es $O(1)$.

P5 — The Adversarial Snake

a) El adversario no cambia el número de copias

Al no tener un adversario, un **resize** solo se ejecuta cuando **size** = **capacity**, y en ese instante se copian exactamente C_j elementos. El adversario controla cuándo ocurre cada **resize**, pero no cuántos ni de qué tamaño (la secuencia de capacidades $C_0, C_1, C_2, \dots$ depende solo de la política y de n). Por tanto, el total de copias $\sum C_j$ es el mismo sin importar el momento del crecimiento.

Para aumentar las copias, el adversario necesitaría poder ejecutar un **resize** con **size** < **capacity**, es decir, controlar también la política de capacidad o contraer la estructura para volver a expandirla. Sin contracción y con política fija, el número de copias queda determinado por n y α .

b–e) Comparación de políticas

- $C + 1$: se copian $2 + j$ en el **resize** j , luego $\sum_{j=0}^{n-3} (2 + j) = \Theta(n^2)$.
- $2C$: serie geométrica de razón 2, $\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2n = O(n)$, y como la última copia es $\Theta(n)$, también $\Omega(n)$. Total $\Theta(n)$.
- $\lceil 1.5C \rceil$: serie geométrica de razón 1.5; el último término domina, $\sum_{j=0}^{r-1} C_j = (C_r - C_0)/(1.5 - 1) = \Theta(n)$.

En los tres casos el costo amortizado por crecimiento es el total de copias dividido entre las n inserciones que provocaron crecimiento. El espacio sin usar tras el **resize** es $C_{\text{new}} - (C_{\text{old}} + 1)$, y su fracción se mide respecto a C_{new} .

Política	# Resizes	Total copias	Costo amortizado	Espacio sin usar tras resize
$C + 1$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	0 (0 %)
$2C$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\approx C/2$ (≈ 50 %)
$\lceil 1.5C \rceil$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\approx C/3$ (≈ 33 %)

f) Trade-off

$C + 1$ ahorra memoria (0 % libre) pero es lento ($\Theta(n^2)$ copias, $\Theta(n)$ amortizado). $2C$ es rápido ($\Theta(1)$ amortizado) pero desperdicia ≈ 50 % de la capacidad. $\lceil 1.5C \rceil$ es el punto intermedio: mismo orden asintótico que $2C$ ($\Theta(\log n)$ **resizes**, $\Theta(n)$ copias, $\Theta(1)$ amortizado) pero con más **resizes** en constante, a cambio de desperdiciar solo ≈ 33 %.

Familia $C_{\text{new}} = \lceil \alpha C \rceil$

Para que la capacidad crezca se necesita $\alpha > 1$. La suma de capacidades es geométrica de razón α ; como $\alpha > 1$, el último término domina:

$$\sum_{j=0}^{r-1} C_j = \frac{C_r - C_0}{\alpha - 1} = \Theta\left(\frac{n}{\alpha - 1}\right), \quad r = \Theta(\log_{\alpha} n).$$

Para α constante y $\alpha > 1$, el total de copias es $\Theta(n/(\alpha - 1)) = \Theta(n)$ y el costo amortizado es $\Theta(1/(\alpha - 1)) = \Theta(1)$. Por tanto, duplicar no es necesario: cualquier $\alpha > 1$ constante funciona. Lo que no funciona es $\alpha = 1 + o(1)$ (crecimiento subgeométrico), porque entonces las copias dejan de ser lineales en n .